

# C / C++ Header Files

## A Beginner's Complete Guide

Learn to write, organise, and use .h files correctly

### 1. What is a Header File?

A **header file** (extension `.h` for C, `.h` or `.hpp` for C++) is a text file that contains **declarations** — descriptions of functions, types, and variables that are defined elsewhere (usually in a matching `.c` or `.cpp` file).

Think of a header file as a *menu in a restaurant*: it tells you what is available and what each item looks like, but the actual cooking happens in the kitchen (the `.c` file).

| Header file (.h)      | Source file (.c / .cpp)        |
|-----------------------|--------------------------------|
| Function declaration  | Function definition (the body) |
| Describes what exists | Contains the actual code       |

|                         |                            |
|-------------------------|----------------------------|
| Shared with other files | Compiled independently     |
| Restaurant menu         | Kitchen where food is made |

## 2. Why Use Header Files?

### Share declarations across many .c files

*Without headers, you would have to copy every function prototype into every file that needs it — error-prone and hard to maintain.*

### Separate interface from implementation

*Users of your library only need the .h file; the .c file can stay private or be compiled into a binary they never read.*

### Enable separate compilation

*Each .c file is compiled independently. Headers let the compiler check types at compile time even when the definition is in a different file.*

### Organise large projects

*A header per module keeps your codebase navigable — you can understand what a module does by reading its header alone.*

## 3. Your First Header File

Here is the smallest complete, correct header file you can write:

```
/* math_utils.h — basic math helpers */

#ifndef MATH_UTILS_H /* include guard — explained in section 4 */
#define MATH_UTILS_H

/* Function declarations (prototypes) */

int add(int a, int b); /* returns a + b */

int subtract(int a, int b); /* returns a - b */

double square_root(double x); /* returns sqrt(x) */
```

```
#endif /* MATH_UTILS_H */
```

And the matching source file:

```
/* math_utils.c */

#include "math_utils.h" /* include our own header first */

#include <math.h> /* for sqrt() */

int add(int a, int b) { return a + b; }

int subtract(int a, int b) { return a - b; }

double square_root(double x) { return sqrt(x); }
```

Using it in another file:

```
/* main.c */

#include <stdio.h>

#include "math_utils.h" /* quotes = look in project folder first */

int main(void) {

    printf("%d\n", add(3, 4)); /* prints 7 */

    printf("%.2f\n", square_root(2.0)); /* prints 1.41 */

    return 0;

}
```

## 4. Include Guards — Never Skip This

If two source files both include the same header, and one also includes the other, a header can get included twice in the same translation unit. This causes **redefinition errors**. Include guards prevent that.

### Method 1 — Traditional `#ifndef` guard (works everywhere)

```
#ifndef MY_HEADER_H /* if not yet defined ... */

#define MY_HEADER_H /* ... define it now (first include only) */

/* all your declarations go here */

#endif /* MY_HEADER_H */
```

**Note:** The macro name must be unique. Convention: `FILENAME_H` in `ALL_CAPS`. For `sensor_driver.h` use `SENSOR_DRIVER_H`.

### Method 2 — `#pragma once` (simpler, supported by all modern compilers)

```
#pragma once /* compiler ensures this file is only included once */

/* all your declarations go here */
```

|                  | <code>#ifndef</code> guard | <code>#pragma once</code>                        |
|------------------|----------------------------|--------------------------------------------------|
| Works everywhere | Yes (C89+)                 | All modern compilers (GCC, Clang, MSVC)          |
| Portable         | 100%                       | Not in the C standard, but universally supported |
| Less to type     | No                         | Yes — one line                                   |
| Faster compile   | Slightly slower            | Compiler can skip file entirely                  |
| Recommended for  | Maximum portability        | Day-to-day development                           |

## 5. What Goes IN a Header File?

**YES**    **Function prototypes (declarations)**

*Tells other files*

```
int calculate_area(int width, int height);
```

**YES**    **Struct and union definitions**

*Types used a*

```
typedef struct { int x; int y; } Point;
```

**YES**    **Enum definitions**

*Enums share*

```
typedef enum { RED, GREEN, BLUE } Color;
```

**YES**    **Macro definitions**

*Constants an*

```
#define MAX_SIZE 256
```

**YES**    **Inline functions (small ones only)**

*Only when th*

```
static inline int max(int a, int b) { return a > b ? a : b; }
```

**YES**    **extern variable declarations**

*Tells other fil*

```
extern int g_error_code;
```

**YES**    **Type aliases**

*Makes types*

```
typedef unsigned char uint8_t;
```

## 6. What Does NOT Belong in a Header?

**Warning: Function definitions (bodies)** — If two .c files include this header, you get a 'multiple definition' linker error.

```
int add(int a, int b) { return a + b; } /* WRONG in .h */
```

**Warning: Non-inline variable definitions** — Defining a variable in a header creates one copy per .c file that includes it.

```
int g_counter = 0; /* WRONG — use extern int g_counter; instead */
```

**Warning: using namespace std; (C++ only)** — It forces every includer to accept that namespace — a common C++ mistake.

```
using namespace std; /* WRONG — pollutes every file that includes yours */
```

**Warning: #include of unnecessary files** — Every unnecessary include slows compilation and creates hidden dependencies.

```
#include <stdio.h> /* only include what the header itself directly needs */
```

## 7. #include Syntax — Angle Brackets vs Quotes

```
#include <stdio.h> /* system / standard library header */

#include <stdlib.h>

#include "my_module.h" /* your own header – search project folder first */

#include "../lib/utils.h" /* relative path works too */
```

| Syntax   | Searches first in                      | Used for                             |
|----------|----------------------------------------|--------------------------------------|
| <file.h> | System include dirs (/usr/include ...) | Standard library, installed packages |
| "file.h" | Same folder as the current file        | Your own project headers             |
| "../x.h" | Relative path from current file        | Headers in parent or sibling folders |

## 8. Structs, Enums, and Typedefs in Headers

Struct and enum **definitions** belong in headers so every file that needs the type can see the full layout.

### Defining a struct with typedef

```
/* person.h */

#ifndef PERSON_H

#define PERSON_H

#include <stdint.h> /* for uint8_t, int32_t ... */

typedef struct {

    char name[64]; /* first and last name */

    uint8_t age; /* 0-255 */

}
```

```

float height_m; /* height in metres */

} Person; /* typedef lets you write Person instead of struct Person */

/* Functions that operate on Person */

void person_init (Person *p, const char *name, uint8_t age);

void person_print (const Person *p);

int person_compare(const Person *a, const Person *b);

#endif /* PERSON_H */

```

## Opaque pointer pattern (hide struct internals)

When you want callers to use a type but *not* see its fields (like a C++ class with private members), declare only a forward reference in the public header and define the full struct in the .c file.

```

/* queue.h – public interface */

#pragma once

typedef struct Queue Queue; /* forward declaration – size is hidden */

Queue *queue_create(void);

void queue_destroy(Queue *q);

void queue_push(Queue *q, int value);

int queue_pop (Queue *q);

```

```

/* queue.c – private implementation */

#include "queue.h"

#include <stdlib.h>

```



```

struct Queue { /* full definition lives only here */

int *data;

int head, tail, capacity;

};

/* ... function bodies ... */

```

## 9. Global Variables Across Files

If you need a global variable visible to multiple files, **declare** it with `extern` in the header and **define** it (once) in one `.c` file.

```

/* globals.h */

#pragma once

extern int g_error_code; /* declaration – no storage allocated */

extern const char *g_app_name;

```

```

/* globals.c */

#include "globals.h"

int g_error_code = 0; /* definition – storage is here */

const char *g_app_name = "MyApp";

```

**Warning:** Never put `int g_error_code = 0;` (without `extern`) in a header. Every `.c` file that includes it would define its own copy — then the linker will report 'multiple definition' errors.

## 10. Macros and Constants

```
/* constants.h */

#pragma once

/* Simple constant macros */

#define MAX_BUFFER_SIZE 1024

#define PI 3.14159265358979f

#define APP_VERSION "2.1.0"

/* Function-like macro (be careful with side effects!) */

#define MIN(a, b) ((a) < (b) ? (a) : (b))

#define MAX(a, b) ((a) > (b) ? (a) : (b))

#define CLAMP(x, lo, hi) ((x) < (lo) ? (lo) : ((x) > (hi) ? (hi) : (x)))

/* Prefer static const or enum for typed constants in C */

static const int MAX_RETRIES = 3;

static const float TIMEOUT_SEC = 30.0f;

/* Enum for named integer constants – safer than bare macros */

typedef enum {

    STATUS_OK = 0,

    STATUS_TIMEOUT = 1,
```

```
STATUS_ERROR = 2,  
  
} StatusCode;
```

**Tip:** Always wrap macro arguments in parentheses: `#define SQUARE(x) ((x)*(x))` not `#define SQUARE(x) x*x`. The second form breaks for `SQUARE(a+1)` — it expands to `a+1*a+1` instead of `(a+1)*(a+1)`.

## 11. Full Working Example — A Stack Module

This example shows a complete, real-world-style C module with a header and source file. The module implements a fixed-size integer stack.

### stack.h — the public interface

```
/* stack.h — fixed-size integer stack  
  
 * Include this header in any file that needs a Stack. */  
  
#pragma once  
  
#include <stddef.h> /* size_t */  
  
#include <stdbool.h> /* bool */  
  
#define STACK_MAX_SIZE 64 /* maximum elements */  
  
/* The Stack type — callers can see the layout (not opaque here) */  
  
typedef struct {  
  
    int data[STACK_MAX_SIZE];  
  
    size_t top; /* index of next free slot */  
  
} Stack;
```

```

/* Lifecycle */

void stack_init (Stack *s); /* initialise / reset */


/* Operations */

bool stack_push (Stack *s, int val); /* false if full */

bool stack_pop (Stack *s, int *out); /* false if empty */

bool stack_peek (const Stack *s, int *out); /* read top without removing */


/* Queries */

bool stack_is_empty(const Stack *s);

bool stack_is_full (const Stack *s);

size_t stack_size (const Stack *s); /* number of elements */

```

## stack.c — the implementation

```

#include "stack.h"

#include <string.h> /* memset */

void stack_init(Stack *s) { memset(s, 0, sizeof(*s)); }

bool stack_push(Stack *s, int val) {

    if (stack_is_full(s)) return false;

    s->data[s->top++] = val;

    return true;

}

```

```

bool stack_pop(Stack *s, int *out) {

    if (stack_is_empty(s)) return false;

    *out = s->data[--s->top];

    return true;

}

bool stack_peek(const Stack *s, int *out) {

    if (stack_is_empty(s)) return false;

    *out = s->data[s->top - 1];

    return true;

}

bool stack_is_empty(const Stack *s) { return s->top == 0; }

bool stack_is_full (const Stack *s) { return s->top == STACK_MAX_SIZE; }

size_t stack_size (const Stack *s) { return s->top; }

```

## 12. Organising Headers in a Project

```
my_project/

include/ # PUBLIC headers – shipped with a library

my_project/ # namespace folder avoids name clashes

stack.h

queue.h

list.h

src/ # source files + PRIVATE headers

stack.c

queue.c

internal.h # private helper – NOT in include/

CMakeLists.txt
```

Putting public headers in a dedicated `include/` folder and private ones next to their `.c` files is the standard layout for C libraries. In CMake:

```
target_include_directories(my_lib

PUBLIC include/ # callers get this search path too

PRIVATE src/ # only this target sees private headers

)
```

## 13. #include Order Convention

Following a consistent include order catches missing dependencies early — if your header forgets to include something it needs, you will find out immediately rather than by accident.

```

/* Recommended include order in a .c file:

* 1. The matching header for this file (self-sufficiency check)

* 2. Standard library headers

* 3. Third-party / system headers

* 4. Other project headers

*/

#include "my_module.h" /* 1. own header – ensures it is self-contained */

#include <stdint.h> /* 2. standard library */

#include <string.h>

#include <stdio.h>

#include <zlib.h> /* 3. third-party */

#include "utils.h" /* 4. other project headers */

#include "config.h"

```

## 14. Common Mistakes and How to Fix Them

| Mistake                                        | Symptom                                                | Fix                                                                                           |
|------------------------------------------------|--------------------------------------------------------|-----------------------------------------------------------------------------------------------|
| Missing include guard                          | Header included twice → redefinition error             | Add <code>#pragma once</code> or <code>#ifndef</code> guard as the first line of every header |
| Defining a function body in a .h               | Linker error: multiple definition of 'foo'             | Move the body to a .c file; keep only the prototype in the .h                                 |
| Defining a variable without extern in header   | Linker error: multiple definition of 'g_var'           | Use <code>extern int g_var;</code> in .h and <code>int g_var = 0;</code> in exactly one .c    |
| Using <> for your own headers                  | <code>#include &lt;my_header.h&gt;;</code> — not found | Use double quotes: <code>#include "my_header.h"</code>                                        |
| Circular includes (A includes B, B includes A) | Infinite recursion → compiler crashes                  | Use forward declarations; restructure so only one direction includes the other                |

|                                                  |                                   |                                                                        |
|--------------------------------------------------|-----------------------------------|------------------------------------------------------------------------|
| Including unnecessary headers                    | Slow compilation, hidden coupling | Include only what the header itself directly uses; move the rest to .c |
| No include guard on a header that types a struct | Type redefinition error           | Every header with a typedef or struct definition needs a guard         |



## 15. Quick Reference Cheat Sheet

| What                        | How to write it                                                   |
|-----------------------------|-------------------------------------------------------------------|
| Include guard (traditional) | <code>#ifndef MY_H / #define MY_H / #endif</code>                 |
| Include guard (modern)      | <code>#pragma once</code>                                         |
| Function prototype          | <code>int foo(int x, float y);</code>                             |
| Struct definition + typedef | <code>typedef struct { int x; } Point;</code>                     |
| Enum definition             | <code>typedef enum { A, B, C } Color;</code>                      |
| Constant macro              | <code>#define MAX_SIZE 256</code>                                 |
| Function-like macro         | <code>#define SQ(x) ((x)*(x))</code>                              |
| Static inline function      | <code>static inline int clamp(int v, int lo, int hi) {...}</code> |
| Global variable declaration | <code>extern int g_count; /* in .h */</code>                      |
| Global variable definition  | <code>int g_count = 0; /* in .c – once */</code>                  |
| Forward struct declaration  | <code>typedef struct Foo Foo;</code>                              |
| Include own header (in .c)  | <code>#include "my_module.h"</code>                               |
| Include system header       | <code>#include &lt;stdio.h&gt;</code>                             |
| Conditional compilation     | <code>#ifdef PLATFORM_ARM ... #endif</code>                       |
| C++ compatibility guard     | <code>#ifdef __cplusplus extern "C" { #endif</code>               |

**Tip: C++ compatibility:** if your C header may be included from C++ code, wrap all declarations in `extern "C" { ... }` guards so the C++ compiler does not mangle function names: `#ifdef __cplusplus extern "C" { #endif` at the top and `#ifdef __cplusplus } #endif` at the bottom.